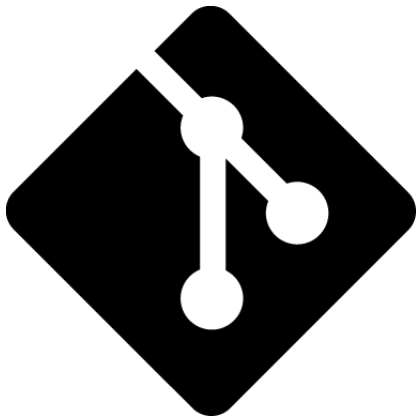




Git Introduction Course

Romain De Groote

February 24, 2026

Why Git?

Git offer many benefits for developers and teams, including:

Why Git?

Git offer many benefits for developers and teams, including:

- ▶ Share code with others
- ▶ Track changes
- ▶ Revert to previous versions
- ▶ Collaborate effectively
- ▶ Designed to be fast, efficient, and scalable
- ▶ Widely used in the software development industry

Why Git?

Git offer many benefits for developers and teams, including:

- ▶ Share code with others
- ▶ Track changes
- ▶ Revert to previous versions
- ▶ Collaborate effectively
- ▶ Designed to be fast, efficient, and scalable
- ▶ Widely used in the software development industry

Why not:

- ▶ Use a shared folder?
- ▶ Use a cloud storage service?
- ▶ USB stick?

Why Git?

Git offer many benefits for developers and teams, including:

- ▶ Share code with others
- ▶ Track changes
- ▶ Revert to previous versions
- ▶ Collaborate effectively
- ▶ Designed to be fast, efficient, and scalable
- ▶ Widely used in the software development industry

Why not:

- ▶ Use a shared folder?
- ▶ Use a cloud storage service?
- ▶ USB stick?

Real difficulty: organization between the members to know who works on what, and to avoid conflicts when merging changes.

Starting up

- ▶ Install Git
 - ▶ www.git-scm.com/downloads

Starting up

- ▶ Install Git
 - ▶ www.git-scm.com/downloads
- ▶ Setting up Git

Starting up

- ▶ Install Git
 - ▶ www.git-scm.com/downloads
- ▶ Setting up Git
- ▶ Create GitHub/GitLab account
 - ▶ GitHub: github.com/join
 - ▶ GitLab: gitlab.com/users/sign_up

Starting up

- ▶ Install Git
 - ▶ www.git-scm.com/downloads
- ▶ Setting up Git
- ▶ Create GitHub/GitLab account
 - ▶ GitHub: github.com/join
 - ▶ GitLab: gitlab.com/users/sign_up
- ▶ (Overleaf)

Starting up

- ▶ Install Git
 - ▶ www.git-scm.com/downloads
- ▶ Setting up Git
- ▶ Create GitHub/GitLab account
 - ▶ GitHub: github.com/join
 - ▶ GitLab: gitlab.com/users/sign_up
- ▶ (Overleaf)

Online systems are not protected from all vulnerabilities, so try to always have a local copy of your most important works.

[GitHub status](#)

[GitLab status](#)

Cloning a repository

Practise exercise:

- ▶ Open Git Bash
- ▶ Use `cd <path>` to navigate to the folder where you want to clone the repository
 - ▶ try `ls` to list the content of the folder and `cd ..` to go back to the parent folder
- ▶ Use `git clone https://github.com/MatsuneMikuroi/GitCrashCourse.git <folder_name>` to clone the repository into a specific folder

Initializing a repository

Practise exercise:

- ▶ Open Git Bash
- ▶ Use `cd <path>` to navigate to the folder where you want to initialize the repository
- ▶ Use `git init` to initialize a new Git repository

Branching

Branches allows you to:

- ▶ Work on different version of a same file

Branching

Branches allows you to:

- ▶ Work on different version of a same file
- ▶ Create a new copy where you can test/experiment

Branching

Branches allows you to:

- ▶ Work on different version of a same file
- ▶ Create a new copy where you can test/experiment
- ▶ Merge the changes if you are satisfied

Branching

Branches allows you to:

- ▶ Work on different version of a same file
- ▶ Create a new copy where you can test/experiment
- ▶ Merge the changes if you are satisfied

The main branch (good practice):

- ▶ Core of the project
- ▶ Stable version only
- ▶ Only merge when you are sure the changes are good

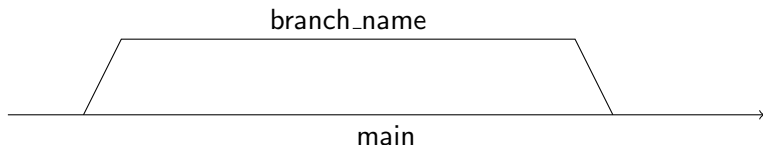
Working with branches

Practise exercise:

- ▶ Go the folder where you initialized the repository
- ▶ Create a .txt file and add some content to it
- ▶ Use `git add <file_name>` to stage the file
- ▶ Use `git commit -m "Initial commit"` to commit the changes
- ▶ Use `git branch <branch_name>` to create a new branch
- ▶ Use `git checkout <branch_name>` to switch to the new branch
- ▶ Make some changes to the .txt file and commit the changes
- ▶ Use `git checkout main` to switch back to the main branch
- ▶ Use `git merge <branch_name>` to merge the changes from the new branch to the main branch
- ▶ Use `git branch -d <branch_name>` to delete the new branch

Working with branches (cont.)

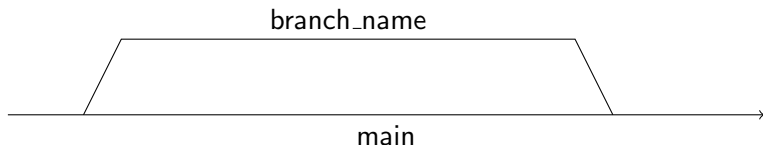
With all this, you should have a Git history that looks like this:



Be cautious, `git merge` merge the selected branch into the current branch, try to avoid this:

Working with branches (cont.)

With all this, you should have a Git history that looks like this:



Be cautious, `git merge` merge the selected branch into the current branch, try to avoid this:



Linking local and remote (IDE and Desktop app)

Github Desktop

If you want to work with Github and have a nice GUI:

- ▶ Download Github Desktop: <https://desktop.github.com/>
- ▶ Link your Github account
- ▶ Clone the repository you created in the previous exercises
- ▶ Make some changes to the .txt file and commit the changes
- ▶ Push the changes to the remote repository
- ▶ Go to the Github website and check that the changes are there

Git Kraken

If you want to work with GitLab/Github/Gitbucket and have a nice GUI:

- ▶ Download Git Kraken: <https://www.gitkraken.com/>
- ▶ Link your Github/GitLab/Gitbucket account
- ▶ Clone the repository you created in the previous exercises
- ▶ Make some changes to the .txt file and commit the changes
- ▶ Push the changes to the remote repository
- ▶ Go to the Github/GitLab/Gitbucket website and check that the changes are there

IDE

In most IDE, you can link your GitHub/GitLab account and work with Git directly from the IDE.

- ▶ Visual Studio Code
- ▶ JetBrains IDEs (PyCharm, IntelliJ, etc.)
- ▶ Eclipse

SSH keys and Personal Access Tokens (PAT)

When you want to push your changes to a remote repository, you need to authenticate yourself.

SSH keys and Personal Access Tokens (PAT)

When you want to push your changes to a remote repository, you need to authenticate yourself.

- ▶ SSH keys
 - ▶ More secure than username/password
 - ▶ Can be used for multiple repositories
 - ▶ Can be used for multiple platforms (Github, GitLab, etc.)

SSH keys and Personal Access Tokens (PAT)

When you want to push your changes to a remote repository, you need to authenticate yourself.

- ▶ SSH keys
 - ▶ More secure than username/password
 - ▶ Can be used for multiple repositories
 - ▶ Can be used for multiple platforms (Github, GitLab, etc.)
- ▶ Personal Access Tokens (PAT)
 - ▶ Used when SSH keys are not supported (e.g., Overleaf)
 - ▶ Can be used for multiple repositories and platforms (Github, GitLab, etc.)
 - ▶ Less secure than SSH keys, but still better than username/password
 - ▶ Can be revoked at any time
 - ▶ Can have specific permissions (e.g., read-only, write-only, etc.)
 - ▶ Can be used for automation (e.g., CI/CD pipelines, scripts, etc.)
 - ▶ Can be used for third-party applications (e.g., Overleaf, etc.)

NEVER SHARE YOUR SSH KEYS
OR PAT WITH ANYONE

NEVER SHARE YOUR SSH KEYS OR PAT WITH ANYONE

- ▶ Scope your PAT at the strict minimum permissions you need
- ▶ If risks of compromise, revoke the PAT/SSH key and generate a new one immediately
- ▶ If you accidentally commit your PAT/SSH key to a Git repository, remove it from the history and revoke it immediately
- ▶ Add collaborators to your repository with the appropriate permissions
- ▶ If you really need to add a PAT/SSH key, add it as a secret

Conclusion

- ▶ Git is a powerful tool for version control and collaboration
- ▶ It is widely used in the software development industry
- ▶ It can be used with different platforms (Github, GitLab, etc.) and tools (IDE, desktop app, etc.)
- ▶ It is important to understand the basics of Git to work effectively with it
- ▶ Always keep a local copy of your most important works and be cautious when sharing your SSH keys or PAT

To go further

- ▶ Markdown files and documentation
- ▶ '.gitignore' files
- ▶ Setting up a Git server locally